

Universidad Autónoma de Santo Domingo

Primada de América | Fundada el 28 de octubre de 1538

Laboratorio de Lenguajes de Programación

Sección 11 (Viernes a las 8pm)

Giancarlo Marte Peralta - 100667358

16/10/2026

Reporte sobre el programa

En esta ocasión, creamos un sistema para rentar vehículos, siguiendo el diagrama UML proporcionado. Para el código, se crearon varios archivos .cs para las distintas clases, primero tenemos una clase abstracta llamada Vehiculos:

```
public abstract class Vehiculo{
    public int Id { get; private set; }
    public string Marca { get; private set; }
    public string Modelo { get; private set; }
    public int Anio { get; private set; }
    public decimal PrecioDiario { get; private set; }
    public EstadoVehiculo Estado { get; private set; }

    public Vehiculo(int id, string marca, string modelo, int anio, decimal precioDiario, EstadoVehiculo estado){
        Id = id;
        Marca = marca;
        Modelo = modelo;
        Anio = anio;
        PrecioDiario = precioDiario;
        Estado = estado;
    }

    public abstract decimal CalcularTarifa(int dias);

    public bool EstaDisponible(DateTime inicio, DateTime fin, List<Reserva> listaReservas){
        if (Estado != EstadoVehiculo.Disponible){
            return false;
        }
        foreach (var reserva in listaReservas){
            if (reserva.Vehiculo.Id == this.Id &&
                (reserva.Estado == EstadoReserva.Pendiente || reserva.Estado == EstadoReserva.Confirmada)){
                bool haySolapamiento = (inicio <= reserva.FechaFin) && (fin >= reserva.FechaInicio);
                if (haySolapamiento){
                    return false;
                }
            }
        }
        return true;
    }

    public void CambiarEstado(EstadoVehiculo nuevoEstado){
        Estado = nuevoEstado;
    }

    public override string ToString(){
        return $"ID: {Id} | {Marca} {Modelo} ({Anio}) | Precio Diario: ${PrecioDiario} | Estado: {Estado}";
    }
}
```

Luego tenemos dos clases derivadas de esta, Automovil y Motocicletas:

```cs
public class Automovil : Vehiculo{
    public int CantidadPuertas { get; private set; }
    public int CapacidadPasajeros { get; private set; }
    public string TipoCombustible { get; private set; }
    public bool TransmisionAutomatica { get; private set; }
    public decimal CapacidadMaletero { get; private set; }

    public Automovil(int id, string marca, string modelo, int anio, decimal precioDiario, EstadoVehiculo estado,
    int cantidadPuertas, int capacidadPasajeros, string tipoCombustible, bool transmisionAutomatica,
    decimal capacidadMaletero) : base(id, marca, modelo, anio, precioDiario, estado){
        CantidadPuertas = cantidadPuertas;
        CapacidadPasajeros = capacidadPasajeros;
        TipoCombustible = tipoCombustible;
        TransmisionAutomatica = transmisionAutomatica;
        CapacidadMaletero = capacidadMaletero;
    }

    public override decimal CalcularTarifa(int dias){
        return PrecioDiario * dias * (TransmisionAutomatica ? 1.15m : 1.0m);
    }
    public decimal CalcularCargoAdicional(){
        return 5.0m;
    }
    public bool ValidarCapacidad(int pasajeros){
        return (pasajeros <= CapacidadPasajeros);
    }
    public override string ToString(){
        string transmision = TransmisionAutomatica ? "Automática" : "Manual";
        return base.ToString() + $" | Puertas: {CantidadPuertas} | Pasajeros: {CapacidadPasajeros} | " +
            $"\nCombustible: {TipoCombustible} | Transmisión: {transmision} | Maletero: {CapacidadMaletero}L";
    }
}
```

```cs
public class Motocicleta : Vehiculo{
    public int Cilindrada { get; private set; }
    public string TipoMotocicleta { get; private set; }
    public int CapacidadCasco { get; private set; }
    public bool TieneMaletero { get; private set; }
    public bool RequiereLicenciaEspecial { get; private set; }

    public Motocicleta(int id, string marca, string modelo, int anio, decimal precioDiario, EstadoVehiculo estado,
    int cilindrada, string tipoMotocicleta, int capacidadCasco, bool tieneMaletero, bool requiereLicenciaEspecial)
    : base(id, marca, modelo, anio, precioDiario, estado){
        Cilindrada = cilindrada;
        TipoMotocicleta = tipoMotocicleta;
        CapacidadCasco = capacidadCasco;
        TieneMaletero = tieneMaletero;
        RequiereLicenciaEspecial = requiereLicenciaEspecial;
    }

    public override decimal CalcularTarifa(int dias){
        return PrecioDiario * dias * 0.9m; // Descuento base para motos
    }

    public bool ValidarLicencia(string tipoLicencia){
        if (RequiereLicenciaEspecial)
            return tipoLicencia == "MotoBasic" || tipoLicencia == "MotoPro";
        return true;
    }

    public decimal CalcularCargoProteccion(){
        return 6.0m;
    }

    public override string ToString(){
        string maletero = TieneMaletero ? "Sí" : "No";
        string licenciaEspecial = RequiereLicenciaEspecial ? "Sí" : "No";

        return base.ToString() + $" | Cilindrada: {Cilindrada}cc | Tipo: {TipoMotocicleta} | " +
            $"\nCascos: {CapacidadCasco} | Maletero: {maletero} | Licencia Especial: {licenciaEspecial}";
    }
}
```

Luego tenemos una clase para Clientes:

```
public class Cliente{
    public int IdCliente { get; private set; }
    public string Nombre { get; private set; }
    public string Documento { get; private set; }
    public string NumeroLicencia { get; private set; }
    public DateTime FechaVencimientoLicencia { get; private set; }
    public string Telefono { get; private set; }


    public Cliente(int idCliente, string nombre, string documento, string numeroLicencia,
    DateTime fechaVencimientoLicencia, string telefono){
        IdCliente = idCliente;
        Nombre = nombre;
        Documento = documento;
        NumeroLicencia = numeroLicencia;
        FechaVencimientoLicencia = fechaVencimientoLicencia;
        Telefono = telefono;
    }


    public bool ValidarLicencia(){
        return FechaVencimientoLicencia > DateTime.Now;
    }

    public bool CumpleEdadMinima(){
        return true;
    }

    public void ActualizarDatos(){
        return;
    }

    public bool PuedeAlquilador(){
        return ValidarLicencia() && CumpleEdadMinima();
    }

    public override string ToString(){
        return $"[ID: {IdCliente}] {Nombre} | Doc: {Documento} | Licencia: {NumeroLicencia} (Vence: {FechaVencimientoLicencia.ToShortDateString()}) | Tel:
{Telefono}";
    }
}
```

También una clase de Reserva, donde se guardan las reservas de vehículos y también se registra el cliente con el que está reservado.

```
public class Reserva{
    public int IdReserva { get; private set; }
    public DateTime FechaInicio { get; private set; }
    public DateTime FechaFin { get; private set; }
    public DateTime FechaCreacion { get; private set; }
    public EstadoReserva Estado { get; private set; }
    public decimal PrecioEstimado { get; private set; }

    public Cliente Cliente { get; private set; }
    public Vehiculo Vehiculo { get; private set; }

    public Reserva(int idReserva, DateTime fechaInicio, DateTime fechaFin, DateTime fechaCreacion, EstadoReserva estado,
    Cliente cliente, Vehiculo vehiculo){
        IdReserva = idReserva;
        FechaInicio = fechaInicio;
        FechaFin = fechaFin;
        FechaCreacion = fechaCreacion;
        Estado = estado;
        Cliente = cliente;
        Vehiculo = vehiculo;
        PrecioEstimado = CalcularPrecioEstimado();
    }

    public int CalcularDuracion() => (FechaFin - FechaInicio).Days;

    public decimal CalcularPrecioEstimado() => Vehiculo.CalcularTarifa(CalcularDuracion());

    public bool Confirmar(List<Reserva> reservas){
        // Regla: La fecha de inicio debe ser anterior a la fecha de fin
        if (FechaInicio >= FechaFin)
            throw new ArgumentException("La fecha de inicio debe ser anterior a la fecha de fin.");

        // Regla: El cliente debe tener licencia vigente
        if (!Cliente.PuedeAlquilador())
            throw new InvalidOperationException("El cliente no cuenta con una licencia vigente o válida.");

        // Regla: Vehículo no puede estar alquilado, en mantenimiento o fuera de servicio
        if (!Vehiculo.EstaDisponible(FechaInicio, FechaFin, reservas))
            throw new InvalidOperationException("El vehículo no está disponible para el período solicitado.");

        Estado = EstadoReserva.Confirmada;
        return true;
    }

    public void Cancelar(){
        Estado = EstadoReserva.Cancelada;
        if (Vehiculo.Estado == EstadoVehiculo.Alquilado){
            Vehiculo.CambiarEstado(EstadoVehiculo.Disponible);
        }
    }

    public override string ToString(){
        return $"Reserva #{IdReserva} | Cliente: {Cliente.Nombre} | Vehículo ID: {Vehiculo.Id} | " +
            $"Del {FechaInicio.ToShortDateString()} al {FechaFin.ToShortDateString()} | " +
            $"Estado: {Estado} | Precio Estimado: ${PrecioEstimado}";
    }
}
```

Y luego una clase para los contratos

```
public class Contrato{
    public int IdContrato { get; private set; }
    public DateTime FechaFirma { get; private set; }
    public DateTime FechaDevolucionPrevista { get; private set; }
    public DateTime? FechaDevolucionReal { get; private set; }
    public decimal MontoTotal { get; private set; }
    public int KilometrajeInicial { get; private set; }
    public int KilometrajeFinal { get; private set; }
    public bool EsActivo { get; private set; }

    public Reserva Reserva { get; private set; }
    private readonly HashSet<string> pagosRegistrados = new HashSet<string>();

    public Contrato(int idContrato, DateTime fechaFirma, DateTime fechaDevolucionPrevista, DateTime? fechaDevolucionReal,
    decimal montoTotal, int kilometrajeInicial, Reserva reserva){
        IdContrato = idContrato;
        FechaFirma = fechaFirma;
        FechaDevolucionPrevista = fechaDevolucionPrevista;
        FechaDevolucionReal = fechaDevolucionReal;
        MontoTotal = montoTotal;
        KilometrajeInicial = kilometrajeInicial;
        Reserva = reserva;
        EsActivo = true;
        KilometrajeFinal = -1;
    }

    public decimal CalcularCargoPorRetraso(){
        if (FechaDevolucionPrevista < (DateTime)FechaDevolucionReal){
            return Reserva.Vehiculo.CalcularTarifa(((DateTime)FechaDevolucionReal - FechaDevolucionPrevista).Days);
        }
        return 0;
    }

    public void CalcularMontoFinal(){
        MontoTotal = Reserva.Vehiculo.CalcularTarifa(Reserva.CalcularDuracion()) + CalcularCargoPorRetraso();
    }

    public void RegistrarPago(string idTransaccionUnica, decimal monto){
        if (monto <= 0)
            throw new ArgumentException("El monto del pago debe ser positivo.");

        if (pagosRegistrados.Contains(idTransaccionUnica))
            throw new InvalidOperationException("Este pago ya ha sido registrado anteriormente en el sistema.");

        pagosRegistrados.Add(idTransaccionUnica);
    }

    public void RegistrarDevolucion(int kilometrajeFinal){
        // Regla: El kilometraje final no puede ser inferior al inicial
        if (kilometrajeFinal < KilometrajeInicial)
            throw new ArgumentException("El kilometraje final no puede ser inferior al kilometraje inicial.");

        FechaDevolucionReal = DateTime.Now;
        CalcularMontoFinal();
        EsActivo = false;
        KilometrajeFinal = kilometrajeFinal;
    }

    public override string ToString(){
        string devRealStr = EsActivo ?  "Pendiente" : FechaDevolucionReal.Value.ToShortDateString();

        return $"Contrato #{IdContrato} | Reserva #{Reserva.IdReserva} | " +
            $"Cliente: {Reserva.Cliente.Nombre} | Vehículo: {Reserva.Vehiculo.Marca} {Reserva.Vehiculo.Modelo} | " +
            $"Firma: {FechaFirma.ToShortDateString()} | Dev. Prevista: {FechaDevolucionPrevista.ToShortDateString()} | " +
            $"Dev. Real: {devRealStr} | Km Inicial: {KilometrajeInicial} km | Monto: ${MontoTotal}";
    }
}
```

Además tenemos unas enumeraciones:

```
public enum EstadoVehiculo { Disponible, Alquilado, Mantenimiento, FueraDeServicio }
public enum EstadoReserva { Pendiente, Confirmada, Cancelada, Finalizada }
```

Y por supuesto el archivo Program que contiene el menú y la mayor parte de la lógica

del programa.

```
List<Reserva> reservas = new List<Reserva>();
List<Contrato> contratos = new List<Contrato>();

// Datos iniciales de prueba para empezar a operar rápido
List<Vehiculo> vehiculos = new List<Vehiculo>{
    // Automóviles (1 al 10)
    new Automovil(1, "Toyota", "Corolla", 2023, 45.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", true, 400),
    new Automovil(2, "Honda", "Civic", 2022, 50.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", false, 450),
    new Automovil(3, "Hyundai", "Elantra", 2024, 40.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", true, 380),
    new Automovil(4, "Kia", "Rio", 2021, 35.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", false, 320),
    new Automovil(5, "Nissan", "Sentra", 2023, 42.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", true, 405),
    new Automovil(6, "Mazda", "3", 2022, 48.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", true, 350),
    new Automovil(7, "Chevrolet", "Cruze", 2020, 38.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", false, 390),
    new Automovil(8, "Ford", "Escape", 2023, 65.0m, EstadoVehiculo.Disponible, 5, 5, "Gasolina", true, 600),
    new Automovil(9, "Toyota", "RAV4", 2024, 75.0m, EstadoVehiculo.Disponible, 5, 5, "Híbrido", true, 580),
    new Automovil(10, "Honda", "CR-V", 2023, 70.0m, EstadoVehiculo.Disponible, 5, 5, "Gasolina", true, 560),

    // Motocicletas (11 al 20)
    new Motocicleta(11, "Yamaha", "MT-03", 2022, 30.0m, EstadoVehiculo.Disponible, 321, "Naked", 1, true, false),
    new Motocicleta(12, "Honda", "CB500F", 2023, 35.0m, EstadoVehiculo.Disponible, 471, "Naked", 1, false, false),
    new Motocicleta(13, "Kawasaki", "Ninja 400", 2023, 40.0m, EstadoVehiculo.Disponible, 399, "Deportiva", 1, false, true),
    new Motocicleta(14, "Suzuki", "V-Strom 650", 2022, 50.0m, EstadoVehiculo.Disponible, 645, "Trail", 2, true, true),
    new Motocicleta(15, "Yamaha", "YZF-R3", 2023, 38.0m, EstadoVehiculo.Disponible, 321, "Deportiva", 1, false, true),
    new Motocicleta(16, "Honda", "Navi", 2024, 20.0m, EstadoVehiculo.Disponible, 109, "Scooter", 1, true, false),
    new Motocicleta(17, "KTM", "Duke 200", 2023, 28.0m, EstadoVehiculo.Disponible, 199, "Naked", 1, false, false),
    new Motocicleta(18, "BMW", "G310R", 2022, 45.0m, EstadoVehiculo.Disponible, 313, "Naked", 1, false, false),
    new Motocicleta(19, "Ducati", "Monster", 2023, 80.0m, EstadoVehiculo.Disponible, 937, "Naked", 1, false, true),
    new Motocicleta(20, "Suzuki", "AX100", 2021, 18.0m, EstadoVehiculo.Disponible, 98, "Urbana", 1, false, false)
};

// --- LISTA DE 20 CLIENTES ---
List<Cliente> clientes = new List<Cliente>{
    new Cliente(101, "Carlos Pérez", "402-1122334-1", "LIC-9988", DateTime.Now.AddYears(2), "809-555-0199"),
    new Cliente(102, "María Gómez", "001-2233445-2", "LIC-1122", DateTime.Now.AddYears(1), "829-555-0288"),
    new Cliente(103, "Juan Rodríguez", "031-3344556-3", "LIC-3344", DateTime.Now.AddYears(3), "849-555-0377"),
    new Cliente(104, "Ana Martínez", "402-4455667-4", "LIC-5566", DateTime.Now.AddYears(2), "809-555-0466"),
    new Cliente(105, "Luis Fernández", "002-5566778-5", "LIC-7788", DateTime.Now.AddYears(1), "829-555-0555"),
    new Cliente(106, "Carmen Sánchez", "003-6677889-6", "LIC-9900", DateTime.Now.AddYears(4), "849-555-0644"),
    new Cliente(107, "Pedro Ramírez", "401-7788990-7", "LIC-2233", DateTime.Now.AddYears(2), "809-555-0733"),
    new Cliente(108, "Lucía Torres", "001-8899001-8", "LIC-4455", DateTime.Now.AddYears(3), "829-555-0822"),
    new Cliente(109, "Sofía Díaz", "037-9900112-9", "LIC-6677", DateTime.Now.AddYears(1), "849-555-0911"),
    new Cliente(110, "Mateo Vargas", "402-0011223-0", "LIC-8899", DateTime.Now.AddYears(2), "809-555-1000"),
    new Cliente(111, "Valentina Castro", "001-1122334-9", "LIC-1010", DateTime.Now.AddYears(3), "829-555-1111"),
    new Cliente(112, "Alejandro Morales", "002-2233445-8", "LIC-2020", DateTime.Now.AddYears(1), "849-555-1222"),
    new Cliente(113, "Camila Herrera", "402-3344556-7", "LIC-3030", DateTime.Now.AddYears(2), "809-555-1333"),
    new Cliente(114, "Gabriel Jiminián", "003-4455667-6", "LIC-4040", DateTime.Now.AddYears(4), "829-555-1444"),
    new Cliente(115, "Isabella Peña", "401-5566778-5", "LIC-5050", DateTime.Now.AddYears(1), "849-555-1555"),
    new Cliente(116, "Daniel Rivas", "001-6677889-4", "LIC-6060", DateTime.Now.AddYears(2), "809-555-1666"),
    new Cliente(117, "Valeria Medina", "031-7788990-3", "LIC-7070", DateTime.Now.AddYears(3), "829-555-1777"),
    new Cliente(118, "Diego Castillo", "402-8899001-2", "LIC-8080", DateTime.Now.AddYears(2), "849-555-1888"),
    new Cliente(119, "Martina Peguero", "002-9900112-1", "LIC-9090", DateTime.Now.AddYears(1), "809-555-1999"),
    new Cliente(120, "Lucas Almonte", "003-0011223-5", "LIC-9991", DateTime.Now.AddYears(3), "829-555-2000")
};

bool salir = false;

while (!salir)
{
    Console.Clear();
    Console.WriteLine("================================================");
    Console.WriteLine("       SISTEMA DE ALQUILER DE VEHÍCULOS         ");
    Console.WriteLine("================================================");
    Console.WriteLine(" 1. Registrar nuevo Vehículo (Auto / Moto)");
    Console.WriteLine(" 2. Registrar nuevo Cliente");
    Console.WriteLine(" 3. Listar Vehículos y Clientes");
    Console.WriteLine(" 4. Crear Reserva");
    Console.WriteLine(" 5. Confirmar Reserva (Genera Cambio de Estado)");
    Console.WriteLine(" 6. Generar Contrato y Ver Detalles");
    Console.WriteLine(" 7. Registrar Devolución de Vehículo");
    Console.WriteLine(" 8. Salir");
```

```
    Console.WriteLine("================================================");
    Console.Write("Seleccione una opción (1-8): ");

    string opcion = Console.ReadLine();
    Console.WriteLine();

    switch (opcion)
    {
        case "1":
            Console.WriteLine("--- REGISTRAR VEHÍCULOS ---");
            Console.Write("¿Es Automóvil (1) o Motocicleta (2)?: ");
            string tipoV = Console.ReadLine();
            Console.Write("ID del vehículo: ");
            int idV = int.Parse(Console.ReadLine());
            Console.Write("Marca: ");
            string marca = Console.ReadLine();
            Console.Write("Modelo: ");
            string modelo = Console.ReadLine();
            Console.Write("Año: ");
            int anio = int.Parse(Console.ReadLine());
            Console.Write("Precio diario ($): ");
            decimal precio = decimal.Parse(Console.ReadLine());

            if (tipoV == "1")
            {
                Console.Write("Cantidad de puertas: ");
                int puertas = int.Parse(Console.ReadLine());
                Console.Write("Capacidad pasajeros: ");
                int pas = int.Parse(Console.ReadLine());
                Console.Write("Combustible: ");
                string comb = Console.ReadLine();
                Console.Write("¿Es transmisión automática? (s/n): ");
                bool aut = Console.ReadLine().ToLower() == "s";
                Console.Write("Capacidad maletero: ");
                decimal mal = decimal.Parse(Console.ReadLine());

                vehiculos.Add(new Automovil(idV, marca, modelo, anio, precio, EstadoVehiculo.Disponible, puertas, pas, comb, aut, mal));
                Console.WriteLine("\n¡Automóvil registrado con éxito!");
            }
            else
            {
                Console.Write("Cilindrada (cc): ");
                int cil = int.Parse(Console.ReadLine());
                Console.Write("Tipo de motocicleta: ");
                string tMoto = Console.ReadLine();
                Console.Write("Capacidad de cascos: ");
                int cascos = int.Parse(Console.ReadLine());
                Console.Write("¿Tiene maletero? (s/n): ");
                bool tieneMal = Console.ReadLine().ToLower() == "s";
                Console.Write("¿Requiere licencia especial? (s/n): ");
                bool licEsp = Console.ReadLine().ToLower() == "s";

                vehiculos.Add(new Motocicleta(idV, marca, modelo, anio, precio, EstadoVehiculo.Disponible, cil, tMoto, cascos, tieneMal, licEsp));
                Console.WriteLine("\n¡Motocicleta registrada con éxito!");
            }
            break;

        case "2":
            Console.WriteLine("--- REGISTRAR CLIENTE ---");
            Console.Write("ID del Cliente: ");
            int idC = int.Parse(Console.ReadLine());
            Console.Write("Nombre completo: ");
            string nom = Console.ReadLine();
            Console.Write("Documento de identidad: ");
            string doc = Console.ReadLine();
            Console.Write("Número de Licencia: ");
            string lic = Console.ReadLine();
            Console.Write("Teléfono: ");
            string tel = Console.ReadLine();

            clientes.Add(new Cliente(idC, nom, doc, lic, DateTime.Now.AddYears(1), tel));
            Console.WriteLine("\n¡Cliente registrado con éxito!");
            break;

        case "3":
            Console.WriteLine("--- LISTADO GENERAL ---");
            Console.WriteLine("\n[VEHÍCULOS]");
            foreach (var v in vehiculos) Console.WriteLine(v.ToString()+"\n");

            Console.WriteLine("\n[CLIENTES]");
```

```
            foreach (var c in clientes) Console.WriteLine(c);
            break;

        case "4":
            Console.WriteLine("--- CREAR RESERVA ---");

            // 1. Primero preguntamos el rango de fechas o los días del alquiler
            Console.Write("Ingrese la fecha de inicio (formato AAAA-MM-DD) o presione Enter para hoy: ");
            string inputInicio = Console.ReadLine();
            DateTime inicio = string.IsNullOrWhiteSpace(inputInicio) ? DateTime.Now : DateTime.Parse(inputInicio);

            Console.Write("¿Cuántos días durará el alquiler?: ");
            int dias = int.Parse(Console.ReadLine());
            DateTime fin = inicio.AddDays(dias);

            // 2. Mostramos los vehículos disponibles filtrados por ese rango de fechas
            MostrarVehiculosDisponibles(vehiculos, inicio, fin, reservas);

            Console.Write("\nIngrese ID del Vehículo a reservar: ");
            int idVehRes = int.Parse(Console.ReadLine());
            Vehiculo vehEncontrado = vehiculos.Find(v => v.Id == idVehRes);

            MostrarClientes(clientes);
            Console.Write("Ingrese ID del Cliente: ");
            int idCliRes = int.Parse(Console.ReadLine());
            Cliente cliEncontrado = clientes.Find(c => c.IdCliente == idCliRes);

            if (vehEncontrado != null && cliEncontrado != null){
                if (vehEncontrado.EstaDisponible(inicio, fin, reservas)){
                    int idRes = reservas.Count + 1;

                    Reserva nuevaReserva = new Reserva(idRes, inicio, fin, DateTime.Now, EstadoReserva.Pendiente, cliEncontrado, vehEncontrado);
                    reservas.Add(nuevaReserva);

                    Console.WriteLine($"\n¡Reserva #{idRes} creada con estado PENDIENTE! Precio estimado: ${nuevaReserva.PrecioEstimado}");
                }
                else{
                    Console.WriteLine("\n[Error] El vehículo seleccionado ya no está disponible para esas fechas.");
                }
            }
            else{
                Console.WriteLine("\n[Error] Vehículo o Cliente no encontrados.");
            }
            break;

        case "5":
            Console.WriteLine("--- CONFIRMAR RESERVA ---");
            MostrarReservasPendientes(reservas);
            Console.Write("Ingrese ID de la Reserva a confirmar: ");
            Reserva resAConf = null;
            while (resAConf == null){
                Console.Write("Ingrese ID de la Reserva a confirmar: ");
                string entrada = Console.ReadLine();
                if (int.TryParse(entrada, out int idResConf)){
                    resAConf = reservas.Find(r => r.IdReserva == idResConf && r.Estado == EstadoReserva.Pendiente);

                    if (resAConf == null){
                        Console.WriteLine("[Error] No se encontró ninguna reserva con ese ID. Intente nuevamente.\n");
                    }
                }
                else{
                    Console.WriteLine("[Error] Por favor, ingrese un número de ID válido.\n");
                }
            }
            bool confirmado = resAConf.Confirmar(reservas.Where(r => r != resAConf).ToList());
            if (confirmado)
                Console.WriteLine("\n¡Reserva confirmada exitosamente!");
            else
                Console.WriteLine("\n[Error] No se pudo confirmar (Verifique si el cliente puede alquilar o si el vehículo está disponible).");
            break;

        case "6":
            Console.WriteLine("--- GENERAR CONTRATO ---");
            MostrarReservasConfirmadas(reservas);
            Console.Write("Ingrese ID de la Reserva Confirmada: ");
            Reserva resParaContrato = null;
            while (resParaContrato == null){
                Console.Write("Ingrese ID de la Reserva a confirmar: ");
                string entrada = Console.ReadLine();
                if (int.TryParse(entrada, out int idResConf)){
```

```
                    resParaContrato = reservas.Find(r => r.IdReserva == idResConf && r.Estado == EstadoReserva.Confirmada);

                    if (resParaContrato == null){
                        Console.WriteLine("[Error] No se encontró ninguna reserva con ese ID. Intente nuevamente.\n");
                    }
                }
                else{
                    Console.WriteLine("[Error] Por favor, ingrese un número de ID válido.\n");
                }
            }
            Console.Write("Ingrese kilometraje inicial del vehículo: ");
            int kmIni = int.Parse(Console.ReadLine());

            int idCon = contratos.Count + 1;
            DateTime fechaFirma = DateTime.Now;
            DateTime fechaDevPrevista = resParaContrato.FechaFin;
            DateTime? fechaDevReal = null;
            decimal montoTotal = resParaContrato.PrecioEstimado;

            Contrato nuevoContrato = new Contrato(idCon, fechaFirma, fechaDevPrevista, fechaDevReal,
            montoTotal, kmIni, resParaContrato);
            contratos.Add(nuevoContrato);
            break;

        case "7":
            Console.WriteLine("--- REGISTRAR DEVOLUCIÓN ---");
            MostrarContratosActivos(contratos);
            Console.Write("Ingrese ID del Contrato activo: ");
            Contrato contratoDev = null;
            while (contratoDev == null){
                Console.Write("Ingrese ID del Contrato: ");
                string entrada = Console.ReadLine();
                if (int.TryParse(entrada, out idCon)){
                    contratoDev = contratos.Find(r => r.IdContrato == idCon && r.EsActivo);

                    if (contratoDev == null){
                        Console.WriteLine("[Error] No se encontró ningún contrato con ese ID. Intente nuevamente.\n");
                    }
                }
                else{
                    Console.WriteLine("[Error] Por favor, ingrese un número de ID válido.\n");
                }
            }
            Console.Write("Ingrese kilometraje final al devolver: ");
            int kmFin = int.Parse(Console.ReadLine());

            contratoDev.RegistrarDevolucion(kmFin);
            Console.WriteLine($"\niDevolución registrada con éxito! Monto final cobrado (incluyendo cargos/retrasos): ${contratoDev.MontoTotal}");
            break;

        case "8":
            salir = true;
            Console.WriteLine("Saliendo del sistema. ¡Gracias por usar la aplicación!");
            break;

        default:
            Console.WriteLine("[Error] Opción no válida.");
            break;
    }

    if (!salir)
    {
        Console.WriteLine("\nPresione cualquier tecla para continuar...");
        Console.ReadKey();
    }
}

static void MostrarVehiculosDisponibles(List<Vehiculo> vehiculos, DateTime inicio, DateTime fin, List<Reserva> reservas){
    Console.WriteLine($"\n--- Vehículos Disponibles del {inicio.ToShortDateString()} al {fin.ToShortDateString()} ---");
    bool hayDisponibles = false;
```

```
    foreach (var v in vehiculos){
        // Comprueba si está libre tanto en estado general como en el rango de fechas y reservas
        if (v.EstaDisponible(inicio, fin, reservas)){
            Console.WriteLine(v.ToString()+"\n");
            hayDisponibles = true;
        }
    }

    if (!hayDisponibles){
        Console.WriteLine("No hay vehículos disponibles para las fechas seleccionadas.");
    }
}

static void MostrarClientes(List<Cliente> clientes){
    Console.WriteLine("\n--- Lista de Clientes ---");
    foreach (var c in clientes){
        Console.WriteLine(c.ToString());
    }
}

static void MostrarReservasPendientes(List<Reserva> reservas){
    Console.WriteLine("\n--- Reservas Pendientes ---");
    foreach (var r in reservas){
        if (r.Estado == EstadoReserva.Pendiente){
            Console.WriteLine(r.ToString());
        }
    }
}
static void MostrarReservasConfirmadas(List<Reserva> reservas){
    Console.WriteLine("\n--- Reservas Pendientes ---");
    foreach (var r in reservas){
        if (r.Estado == EstadoReserva.Confirmada){
            Console.WriteLine(r.ToString());
        }
    }
}
static void MostrarContratosActivos(List<Contrato> contratos){
    Console.WriteLine("\n--- Contratos Activos ---");
    foreach (var c in contratos){
        if (c.EsActivo){
            Console.WriteLine(c.ToString());
        }
    }
}
```

Por último, tenemos los casos de prueba:

```csharp
using Xunit;
using System;
using System.Collections.Generic;

public class SistemaAlquilerTests{
    [Fact]
    public void Test_Vehiculo_Disponibilidad_SinSolapamientos(){
        // Arrange
        // Inicializamos un automóvil (ajusta los parámetros según tus constructores)
        Automovil auto = new Automovil(1, "Toyota", "Corolla", 2023, 45.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", true, 400);
        List<Reserva> reservas = new List<Reserva>();

        DateTime inicioConsulta = DateTime.Today.AddDays(2);
        DateTime finConsulta = DateTime.Today.AddDays(5);

        // Act
        bool estaDisponible = auto.EstaDisponible(inicioConsulta, finConsulta, reservas);

        // Assert
        Assert.True(estaDisponible, "El vehículo debería estar disponible cuando no hay reservas registradas.");
    }

    [Fact]
    public void Test_Tarifa_Polimorfica(){
        // Arrange
        // Verificamos el comportamiento polimórfico del cálculo de tarifas según el tipo de vehículo
        Vehiculo auto = new Automovil(6, "Mazda", "3", 2022, 48.0m, EstadoVehiculo.Disponible, 4, 5, "Gasolina", true, 350);
        Vehiculo moto = new Motocicleta(11, "Yamaha", "MT-03", 2022, 30.0m, EstadoVehiculo.Disponible, 321, "Naked", 1, true, false);
        int diasAlquiler = 3;

        // Act
        decimal tarifaAuto = auto.CalcularTarifa(diasAlquiler);
        decimal tarifaMoto = moto.CalcularTarifa(diasAlquiler);

        // Assert
        // Comprobamos que ambas calculan correctamente de forma independiente (polimorfismo)
        Assert.True(tarifaAuto > 0);
        Assert.True(tarifaMoto > 0);
        Assert.NotEqual(tarifaAuto, tarifaMoto);
    }

    [Fact]
    public void Test_Devolucion_Y_CierreContrato(){
        // Arrange
        Cliente cliente = new Cliente(106, "Carmen Sánchez", "003-6677889-6", "LIC-9900", DateTime.Now.AddYears(4), "849-555-0644");
        Vehiculo auto = new Automovil(8, "Ford", "Escape", 2023, 65.0m, EstadoVehiculo.Disponible, 5, 5, "Gasolina", true, 600);

        DateTime inicio = DateTime.Today.AddDays(-5);
        DateTime fin = DateTime.Today.AddDays(-1);
        Reserva reserva = new Reserva(1, inicio, fin, DateTime.Now.AddDays(-6), EstadoReserva.Confirmada, cliente, auto);

        Contrato contrato = new Contrato(1, inicio, fin, null, 200m, 15000, reserva);

        int kmInicial = contrato.KilometrajeInicial;
        int kmFinal = 15500;

        // Act
        // Simulamos la devolución registrando el kilometraje final y finalizando el contrato/vehículo
        contrato.RegistrarDevolucion(kmFinal); // O el método que hayas definido para la devolución

        // Assert
        Assert.Equal(kmFinal, contrato.KilometrajeFinal);
        Assert.Equal(EstadoVehiculo.Disponible, auto.Estado); // El vehículo debe quedar libre tras la devolución
        Assert.True(!contrato.EsActivo);
    }
}
```

Al correr estos Unit Test tenemos:

```
PS >dotnet test
Restore complete (0.6s)
  ProgramaEnClase net10.0 succeeded with 1 warning(s) (1.3s) → bin\Debug\net10.0\ProgramaEnClase.dll
    C:\Users\zlaae\.nuget\packages\microsoft.net.test.sdk\18.11.0\build\net8.0\Microsoft.NET.Test.Sdk.Program.cs(3,41): warning CS702
2: The entry point of the program is global code; ignoring 'AutoGeneratedProgram.Main(string[])' entry point.
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v4.0.1+0f7d528729 (64-bit .NET 10.0.13)
[xUnit.net 00:00:00.18]   Discovering: ProgramaEnClase
[xUnit.net 00:00:00.26]   Discovered:  ProgramaEnClase
[xUnit.net 00:00:00.36]   Starting:    ProgramaEnClase
[xUnit.net 00:00:00.57]   Finished:    ProgramaEnClase
  ProgramaEnClase test net10.0 succeeded (2.6s)

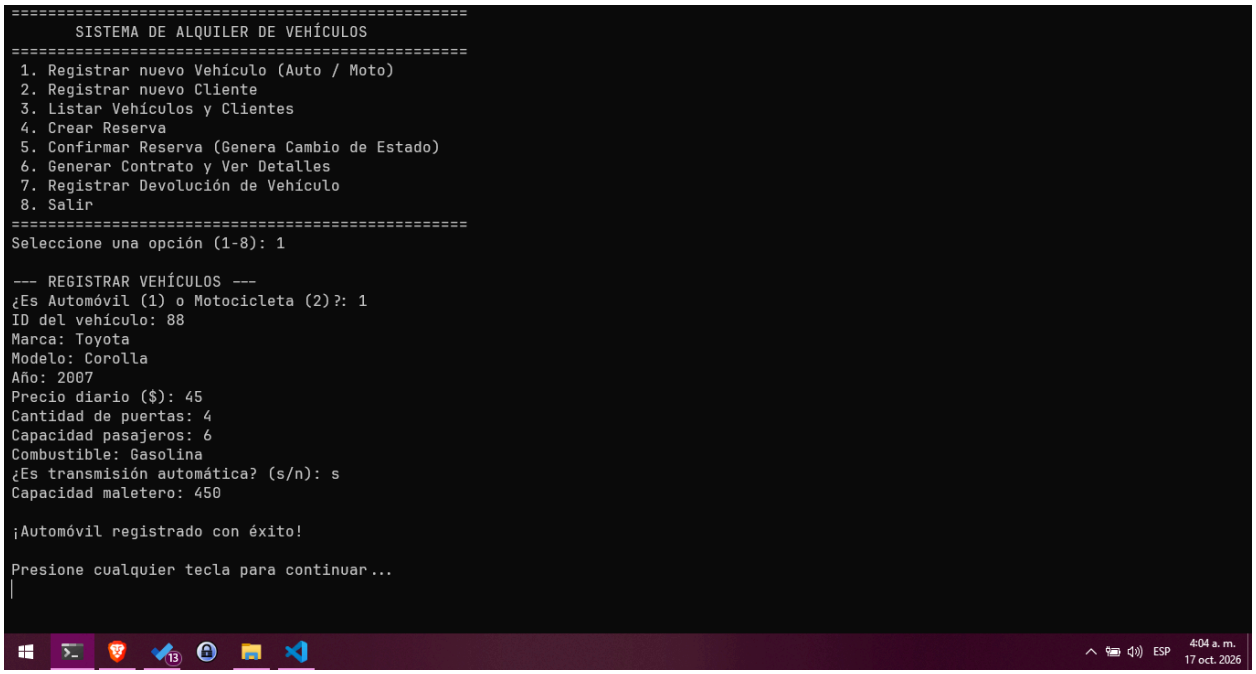
Test summary: total: 3; failed: 0; succeeded: 3; skipped: 0; duration: 2.5s
Build succeeded with 1 warning(s) in 5.6s
PS >
```

Por lo que se aprueban. Ahora bien, al correr el programa tenemos las siguientes posibilidades:

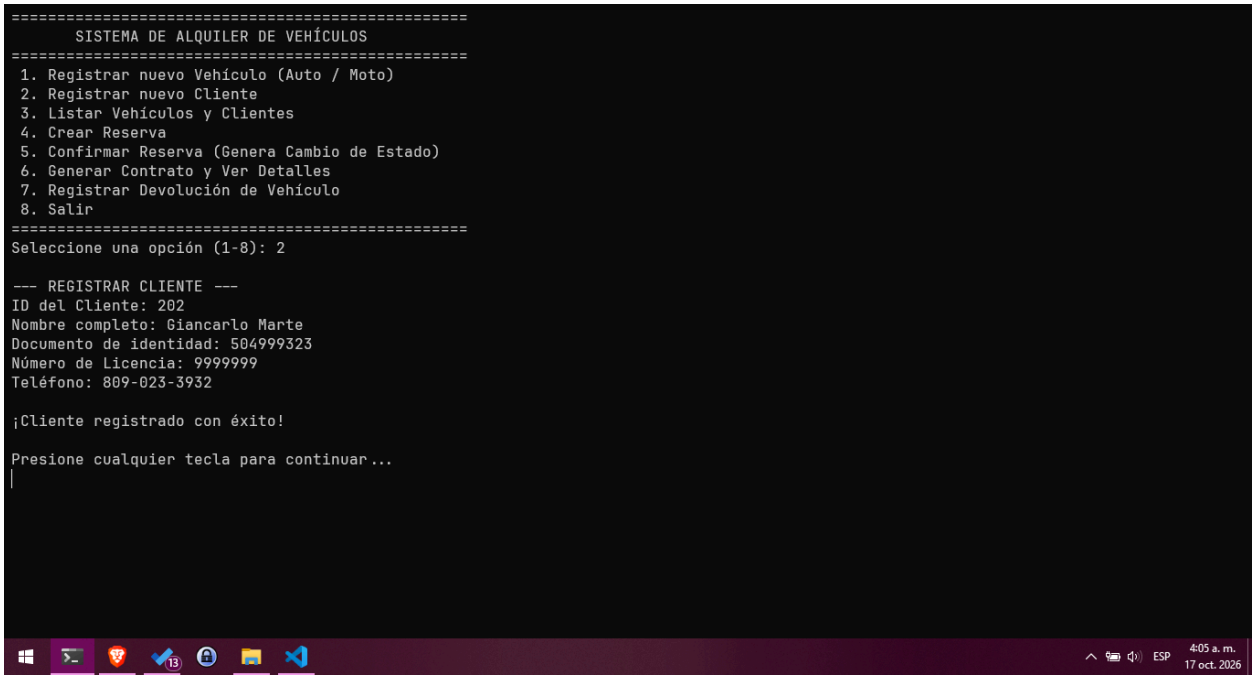
Al compilar y ejecutar nos aparece el menú:

```
==================================================
       SISTEMA DE ALQUILER DE VEHÍCULOS
==================================================
 1. Registrar nuevo Vehículo (Auto / Moto)
 2. Registrar nuevo Cliente
 3. Listar Vehículos y Clientes
 4. Crear Reserva
 5. Confirmar Reserva (Genera Cambio de Estado)
 6. Generar Contrato y Ver Detalles
 7. Registrar Devolución de Vehículo
 8. Salir
==================================================
Seleccione una opción (1-8):
```

Para la opción 1, podemos ver lo siguiente:



En la opción 2 vemos algo similar



En la opción 3 podemos ver los vehículos y clientes registrados:

```
ID: 17 | KTM Duke 200 (2023) | Precio Diario: $28.0 | Estado: Disponible | Cilindrada: 199cc | Tipo: Naked |
Cascos: 1 | Maletero: No | Licencia Especial: No

ID: 18 | BMW G310R (2022) | Precio Diario: $45.0 | Estado: Disponible | Cilindrada: 313cc | Tipo: Naked |
Cascos: 1 | Maletero: No | Licencia Especial: No

ID: 19 | Ducati Monster (2023) | Precio Diario: $80.0 | Estado: Disponible | Cilindrada: 937cc | Tipo: Naked |
Cascos: 1 | Maletero: No | Licencia Especial: Sí

ID: 20 | Suzuki AX100 (2021) | Precio Diario: $18.0 | Estado: Disponible | Cilindrada: 98cc | Tipo: Urbana |
Cascos: 1 | Maletero: No | Licencia Especial: No

ID: 88 | Toyota Corolla (2007) | Precio Diario: $45 | Estado: Disponible | Puertas: 4 | Pasajeros: 6 |
Combustible: Gasolina | Transmisión: Automática | Maletero: 450L


[CLIENTES]
[ID: 101] Carlos Pérez | Doc: 402-1122334-1 | Licencia: LIC-9988 (Vence: 16 oct. 2028) | Tel: 809-555-0199
[ID: 102] María Gómez | Doc: 001-2233445-2 | Licencia: LIC-1122 (Vence: 16 oct. 2027) | Tel: 829-555-0288
[ID: 103] Juan Rodríguez | Doc: 031-3344556-3 | Licencia: LIC-3344 (Vence: 16 oct. 2029) | Tel: 849-555-0377
[ID: 104] Ana Martínez | Doc: 402-4455667-4 | Licencia: LIC-5566 (Vence: 16 oct. 2028) | Tel: 809-555-0466
[ID: 105] Luis Fernández | Doc: 002-5566778-5 | Licencia: LIC-7788 (Vence: 16 oct. 2027) | Tel: 829-555-0555
[ID: 106] Carmen Sánchez | Doc: 003-6677889-6 | Licencia: LIC-9900 (Vence: 16 oct. 2030) | Tel: 849-555-0644
[ID: 107] Pedro Ramírez | Doc: 401-7788990-7 | Licencia: LIC-2233 (Vence: 16 oct. 2028) | Tel: 809-555-0733
[ID: 108] Lucía Torres | Doc: 001-8899001-8 | Licencia: LIC-4455 (Vence: 16 oct. 2029) | Tel: 829-555-0822
[ID: 109] Sofía Díaz | Doc: 037-9900112-9 | Licencia: LIC-6677 (Vence: 16 oct. 2027) | Tel: 849-555-0911
[ID: 110] Mateo Vargas | Doc: 402-0011223-0 | Licencia: LIC-8899 (Vence: 16 oct. 2028) | Tel: 809-555-1000
[ID: 111] Valentina Castro | Doc: 001-1122334-9 | Licencia: LIC-1010 (Vence: 16 oct. 2029) | Tel: 829-555-1111
[ID: 112] Alejandro Morales | Doc: 002-2233445-8 | Licencia: LIC-2020 (Vence: 16 oct. 2027) | Tel: 849-555-1222
[ID: 113] Camila Herrera | Doc: 402-3344556-7 | Licencia: LIC-3030 (Vence: 16 oct. 2028) | Tel: 809-555-1333
[ID: 114] Gabriel Jiminián | Doc: 003-4455667-6 | Licencia: LIC-4040 (Vence: 16 oct. 2030) | Tel: 829-555-1444
[ID: 115] Isabella Peña | Doc: 401-5566778-5 | Licencia: LIC-5050 (Vence: 16 oct. 2027) | Tel: 849-555-1555
```



En la opción 4 podemos crear una reserva

```
Ingrese ID del Vehículo a reservar: 88

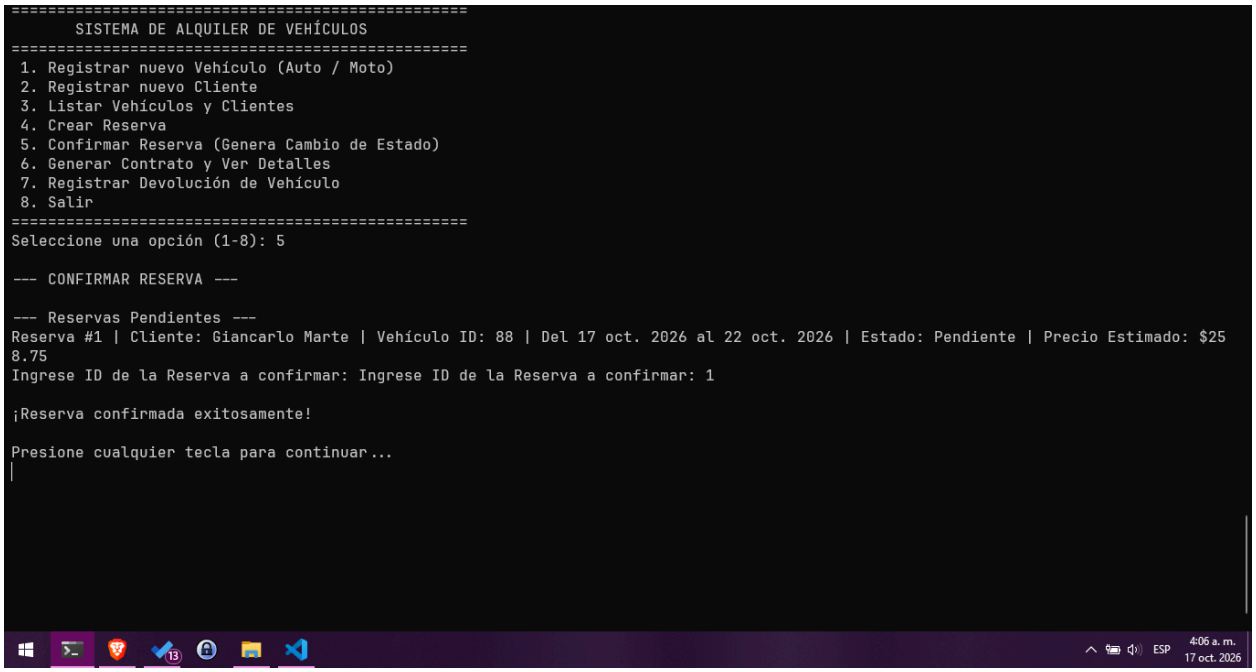
--- Lista de Clientes ---
[ID: 101] Carlos Pérez | Doc: 402-1122334-1 | Licencia: LIC-9988 (Vence: 16 oct. 2028) | Tel: 809-555-0199
[ID: 102] María Gómez | Doc: 001-2233445-2 | Licencia: LIC-1122 (Vence: 16 oct. 2027) | Tel: 829-555-0288
[ID: 103] Juan Rodríguez | Doc: 031-3344556-3 | Licencia: LIC-3344 (Vence: 16 oct. 2029) | Tel: 849-555-0377
[ID: 104] Ana Martínez | Doc: 402-4455667-4 | Licencia: LIC-5566 (Vence: 16 oct. 2028) | Tel: 809-555-0466
[ID: 105] Luis Fernández | Doc: 002-5566778-5 | Licencia: LIC-7788 (Vence: 16 oct. 2027) | Tel: 829-555-0555
[ID: 106] Carmen Sánchez | Doc: 003-6677889-6 | Licencia: LIC-9900 (Vence: 16 oct. 2030) | Tel: 849-555-0644
[ID: 107] Pedro Ramírez | Doc: 401-7788990-7 | Licencia: LIC-2233 (Vence: 16 oct. 2028) | Tel: 809-555-0733
[ID: 108] Lucía Torres | Doc: 001-8899001-8 | Licencia: LIC-4455 (Vence: 16 oct. 2029) | Tel: 829-555-0822
[ID: 109] Sofía Díaz | Doc: 037-9900112-9 | Licencia: LIC-6677 (Vence: 16 oct. 2027) | Tel: 849-555-0911
[ID: 110] Mateo Vargas | Doc: 402-0011223-0 | Licencia: LIC-8899 (Vence: 16 oct. 2028) | Tel: 809-555-1000
[ID: 111] Valentina Castro | Doc: 001-1122334-9 | Licencia: LIC-1010 (Vence: 16 oct. 2029) | Tel: 829-555-1111
[ID: 112] Alejandro Morales | Doc: 002-2233445-8 | Licencia: LIC-2020 (Vence: 16 oct. 2027) | Tel: 849-555-1222
[ID: 113] Camila Herrera | Doc: 402-3344556-7 | Licencia: LIC-3030 (Vence: 16 oct. 2028) | Tel: 809-555-1333
[ID: 114] Gabriel Jiminián | Doc: 003-4455667-6 | Licencia: LIC-4040 (Vence: 16 oct. 2030) | Tel: 829-555-1444
[ID: 115] Isabella Peña | Doc: 401-5566778-5 | Licencia: LIC-5050 (Vence: 16 oct. 2027) | Tel: 849-555-1555
[ID: 116] Daniel Rivas | Doc: 001-6677889-4 | Licencia: LIC-6060 (Vence: 16 oct. 2028) | Tel: 809-555-1666
[ID: 117] Valeria Medina | Doc: 031-7788990-3 | Licencia: LIC-7070 (Vence: 16 oct. 2029) | Tel: 829-555-1777
[ID: 118] Diego Castillo | Doc: 402-8899001-2 | Licencia: LIC-8080 (Vence: 16 oct. 2028) | Tel: 849-555-1888
[ID: 119] Martina Peguero | Doc: 002-9900112-1 | Licencia: LIC-9090 (Vence: 16 oct. 2027) | Tel: 809-555-1999
[ID: 120] Lucas Almonte | Doc: 003-0011223-5 | Licencia: LIC-9991 (Vence: 16 oct. 2029) | Tel: 829-555-2000
[ID: 202] Giancarlo Marte | Doc: 504999323 | Licencia: 9999999 (Vence: 17 oct. 2027) | Tel: 809-023-3932
Ingrese ID del Cliente: 202

¡Reserva #1 creada con estado PENDIENTE! Precio estimado: $258.75

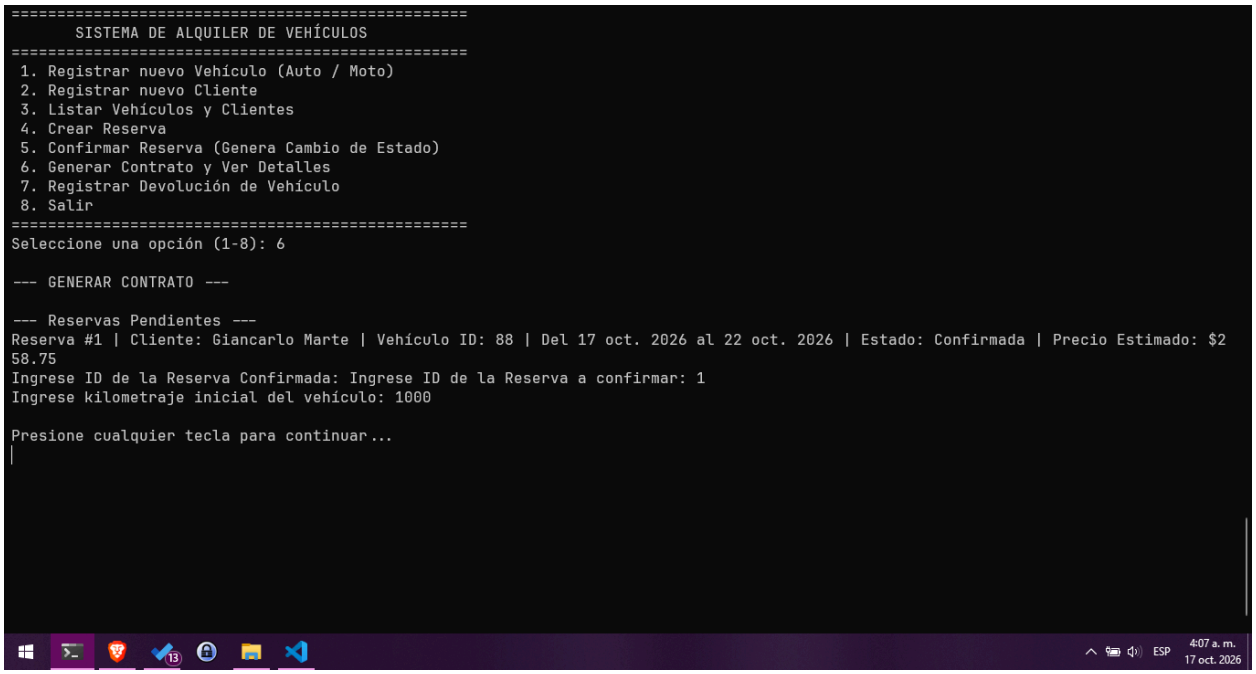
Presione cualquier tecla para continuar...
```



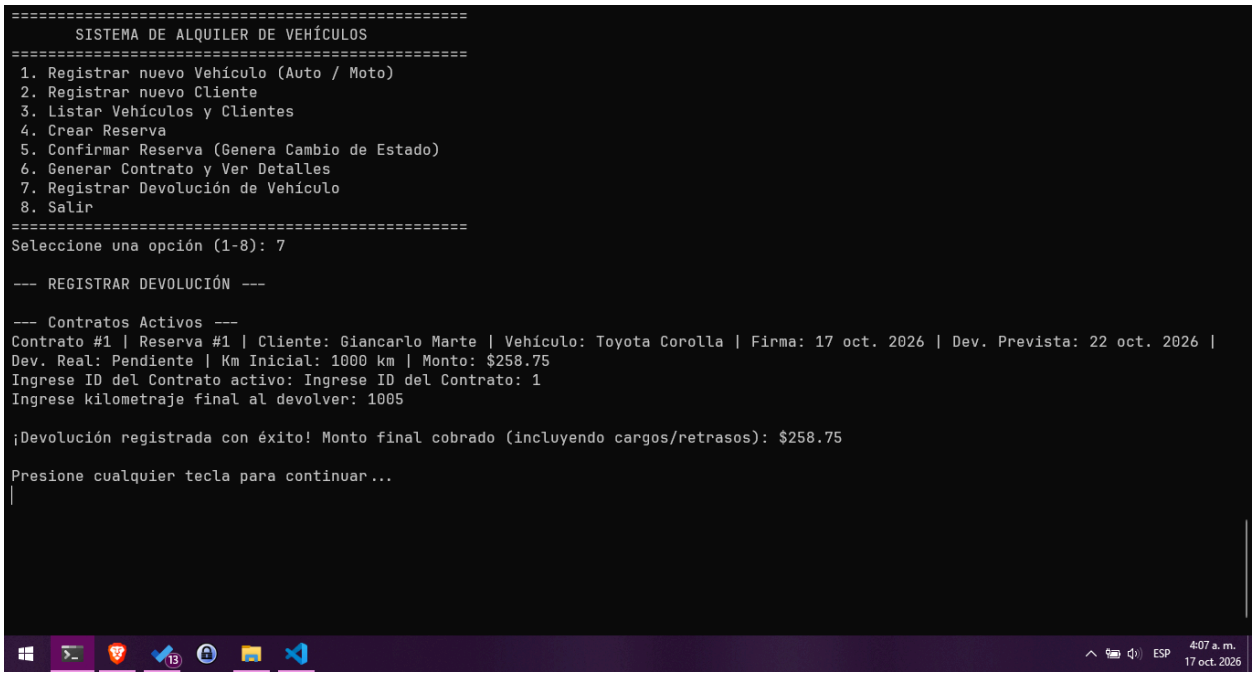
En la opción 5 podemos confirmar la reserva



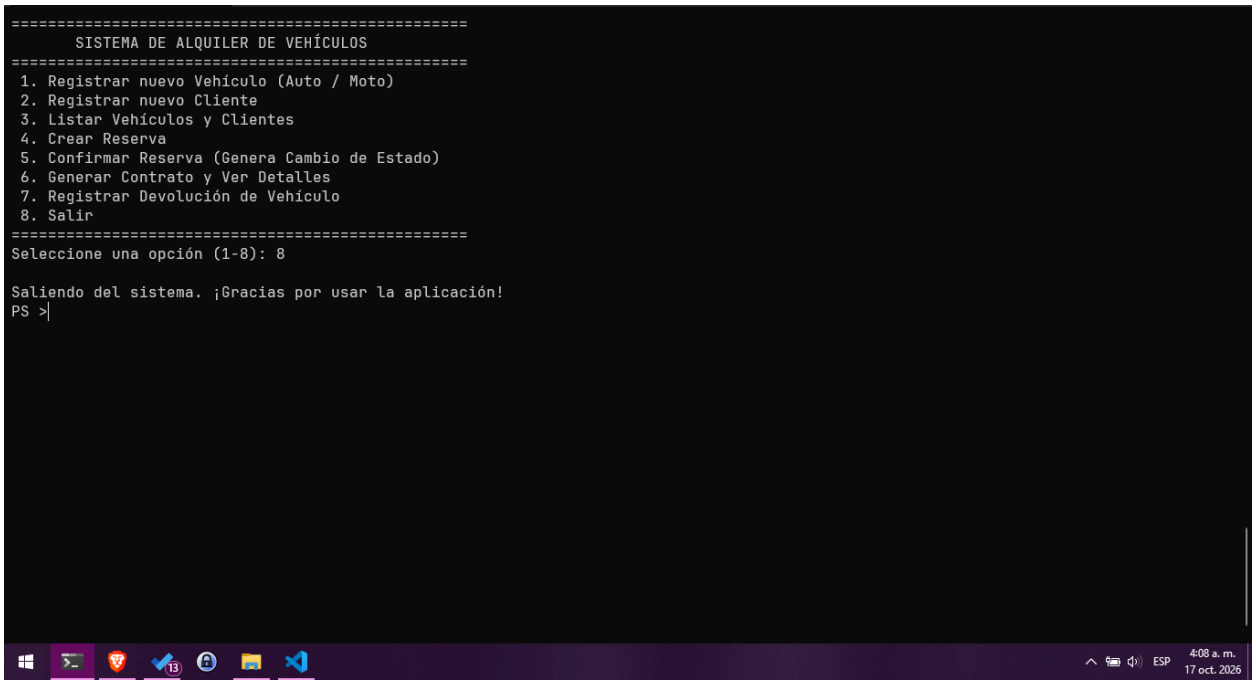
En la opción 6 podemos generar un contrato



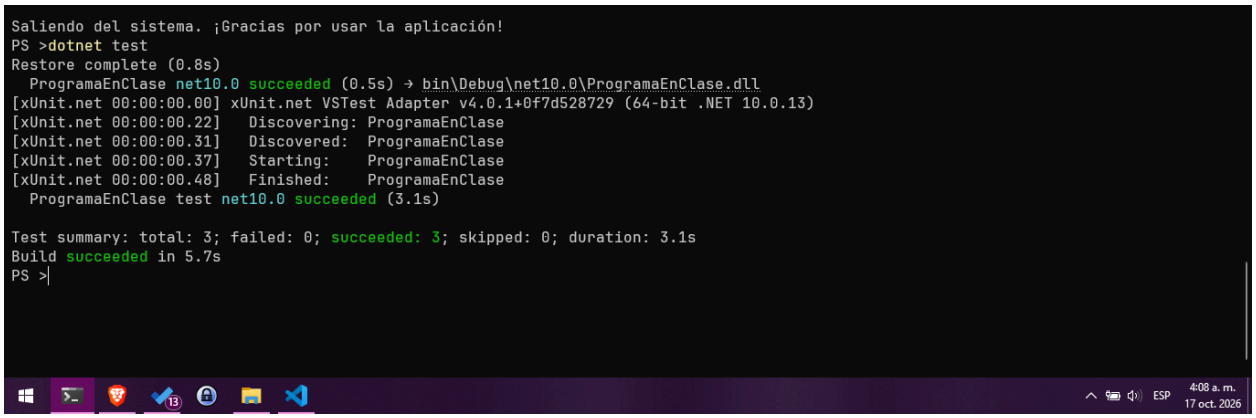
En la opción 7 podemos entonces Registrar la Devolución del Vehículo.



Por último en l opción 8 podemos salir del programa:



Además al correr los tests vemos que todos pasan:


Saliendo del sistema. ¡Gracias por usar la aplicación!
PS >dotnet test
Restore complete (0.8s)
ProgramaEnClase net10.0 succeeded (0.5s) → bin\Debug\net10.0\ProgramaEnClase.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v4.0.1+0f7d528729 (64-bit .NET 10.0.13)
[xUnit.net 00:00:00.22] Discovering: ProgramaEnClase
[xUnit.net 00:00:00.31] Discovered: ProgramaEnClase
[xUnit.net 00:00:00.37] Starting: ProgramaEnClase
[xUnit.net 00:00:00.48] Finished: ProgramaEnClase
ProgramaEnClase test net10.0 succeeded (3.1s)
Test summary: total: 3; failed: 0; succeeded: 3; skipped: 0; duration: 3.1s
Build succeeded in 5.7s
PS >
ESP
4:08 a. m.
17 oct. 2026